



Feature selection optimization algorithm based on evolutionary Q-learning

Guan Yang, Zhiyong Zeng, Xinrui Pu [✉], Ren Duan ^{*}

Yunnan University of Finance and Economics, 237 Longquan Road, Kunming, 650221, Yunnan, China

ARTICLE INFO

Keywords:

Feature selection

Q-learning

PSO

Evolutionary optimization

Multiagent

ABSTRACT

Classification problems are an important research area in the field of data mining and machine learning. To enhance classification accuracy and optimize the effectiveness of learning algorithms, feature selection, as a data preprocessing operation, deserves ongoing attention. Based on reinforcement learning and particle swarm optimization, this paper proposes an evolutionary Q-learning feature selection optimization algorithm (EQL-FS). It leverages the advantages of reinforcement learning and combines them with the global exploration capability of the particle swarm optimization algorithm to achieve the optimal strategy. The multiagent approach is adopted, and the interaction is achieved through particle swarm optimization. The effectiveness of the proposed algorithm has been validated using sixteen public datasets. The experimental results indicate that this new algorithm can select the shortest feature subset without compromising accuracy. Additionally, it demonstrates a robust ability to eliminate noise and redundant features. Furthermore, the algorithm has been applied to analyze the broadband customer base churn for a communication operator, and the results are consistent with those obtained from the public datasets. Finally, the statistical test comparing different algorithms has been completed, and the results indicate that the new algorithm EQL-FS demonstrates statistical significance in terms of accuracy and the number of selected features.

1. Introduction

Classification is a crucial area of research in the field of data mining and machine learning. Its purpose is to accurately categorize each instance into distinct groups based on the information provided by the features. However, as the size of the collected dataset increases, there is a significant increase in irrelevant and redundant feature information. This not only adds complexity to model construction but also hampers the performance of machine learning algorithms. It slows down the learning speed of algorithms and can even lead to issues such as “dimensional disaster” and “overfitting”. It affects the accuracy of classification predictions. Feature selection is an effective data preprocessing technique that has been widely studied in the field of machine learning and pattern recognition. As one of the important methods for dimensionality reduction, feature selection can effectively eliminate irrelevant and redundant feature information from the target task. This approach reduces the computational complexity of machine learning algorithms while enhancing the classification accuracy and generalization capability of the developed models. Additionally, it lowers learning costs and optimizes performance metrics. Feature selection is widely used in bioinformatics, text classification, image processing, and other fields due to its interpretability and ability to preserve original features. This technique assists in understanding the

^{*} Corresponding author.

E-mail address: duanren@ynufe.edu.cn (R. Duan).

<https://doi.org/10.1016/j.ins.2025.122441>

Received 1 December 2023; Received in revised form 15 June 2025; Accepted 16 June 2025

Available online 26 June 2025

0020-0255/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

causal relationship between features and data. With the advent of the big data era, feature selection encounters numerous challenges. These challenges include dataset imbalance and excessively high dimensionality of data features, which traditional feature selection optimization algorithms struggle to address. Consequently, optimizing feature selection algorithms holds significant research and practical application value [1].

Moran [2] proposed a new algorithm for curiosity feature selection (CFS) based on a curiosity loop architecture. Unlike other feature selection methods, this approach can be utilized for both batch learning and online learning. It is not restricted to a particular predictor and offers additional benefits such as runtime efficiency and adaptability to changes in data. In this feature selection mechanism, the agent autonomously learns and selects a set of actions (features) based on the method of reinforcement learning. It uses an internal reward system to maximize the accuracy of the learning model. The algorithm is evaluated on three datasets, and the results show that the features selected using the CFS strategy are better than those selected using standard feature selection methods, resulting in higher accuracy. However, the dataset utilized in this algorithm comprises a limited number of features, while real-world datasets typically exhibit higher dimensionality. Consequently, it is essential to investigate whether the CFS algorithm can still effectively select a relevant subset of features within a restricted number of exploration steps when applied to higher-dimensional datasets. In addition, in the low-dimensional feature space, this method usually begins learning from the first episodes. The process needed to select the optimal feature subset or achieve convergence is relatively lengthy, resulting in low exploration efficiency. Furthermore, when dealing with high-dimensional feature spaces, the issue of low exploration efficiency becomes more apparent.

In recent years, evolutionary reinforcement learning (ERL) has emerged as a hybrid algorithm aimed at optimizing the parameters of policy networks, which are often high-dimensional and challenging for traditional evolutionary algorithms. These hybrid approaches, which integrate reinforcement learning with evolutionary optimization, are well-suited for continuous motion tasks, such as those encountered in MuJoCo. The combination of reinforcement learning and evolutionary optimization algorithms offers distinct advantages. Reinforcement learning automates feature selection, while evolutionary optimization algorithms possess superior global search capabilities. This integrated approach represents a novel algorithm for enhancing the classification performance of high-dimensional datasets.

Based on the literature [2], this paper proposes a new feature selection algorithm called EQL-FS. The algorithm combines reinforcement learning and evolutionary optimization techniques, utilizing Q-learning for reinforcement learning and PSO for evolutionary optimization. Therefore, this algorithm is referred to as evolutionary Q-learning feature selection (EQL-FS). Since it is difficult for CFS to explore the optimal strategy in a limited number of exploration steps when the feature dimension is relatively high and the evolutionary optimization algorithm has strong exploration and optimization abilities, the particle swarm optimization algorithm is integrated with CFS to enhance the optimal subset search ability of reinforcement learning methods in feature selection. In this paper, we select sixteen public datasets to validate the proposed algorithm. The optimal strategy derived from our algorithm shows that it improves exploration efficiency and delivers improved results in high-dimensional feature spaces. In addition, it demonstrates a robust ability to eliminate noise and redundant features. Furthermore, the algorithm has been applied to analyze the broadband customer base churn for a communication operator, and the results are consistent with those obtained from the public datasets.

This paper is organized in the following manner. First, we discuss related work on traditional feature selection, as well as feature selection methods based on evolutionary optimization and reinforcement learning in Section 2. A description of the problem is provided in Section 3. Then, in Section 4, we outline the proposed EQL-FS algorithm. Sections 5 and 6 present experimental results on public and industrial datasets. Section 6 also describes some sensitivity experiments. Section 7 deals with the signed rank test. Section 8 is the conclusion and discussion. The last section gives direction for future work.

2. Related work

2.1. Traditional feature selection

Some of the feature selection algorithms can be characterized as either “embedded”, “filter”, or “wrapper” approaches, based on the relationship between the selection scheme and the basic induction algorithm [3]. (1) Filtering methods typically assess the importance of each feature individually based on specific measurement strategies and then rank them accordingly [4][5]. Because the filter method is independent of the classifier, it offers faster execution speed. However, it does not consider the correlation between features, which may result in unsatisfactory classification performance when selecting feature subsets. Considering the shortcomings of the filter method, several improved algorithms have been successively proposed, such as the iterative local Fisher score (ILFS) [6] and the generalized Fisher score [7]. (2) The wrapper method differs from the filtering method. The wrapper method directly evaluates the performance of the selected learning algorithm as the criterion for evaluating the feature subset. It continuously searches for the optimal feature subset using various search strategies. Due to the need to train the classifier multiple times during the feature selection process, the computational cost is relatively high [8]. (3) The embedded feature selection method integrates the feature selection process into the learning algorithm process. This method relies too heavily on the design of the classifier, resulting in poor scalability of the obtained results [9].

2.2. Feature selection based on reinforcement learning

The traditional approach to feature selection is to consider each feature in a fixed context, either independently or together with all other features, using a filtered and embedded approach. Only wrapper methods that consider feature subsets have a chance of

identifying the best features, but this comes at the expense of evaluating a large number of subsets. To address the limitations of wrapper methods, Romaric Gaudel initially formulated the feature selection problem as a reinforcement learning (RL) problem [10][11]. Many scholars have proposed reinforcement learning methods for feature selection, such as a dynamic feature selection algorithm [12] and Q-learning with Fisher score (QLFS), a feature selection method for large-scale datasets [13]. Among the components of reinforcement learning, another important component is the agent [14][15]. In feature selection, single agents [2][15], and multiagents [14][16] can be selected.

2.3. Feature selection based on evolutionary optimization and reinforcement learning

Feature selection methods developed based on intelligent optimization algorithms can quickly find high-quality feature subsets through global search strategies. As a result, these methods have garnered significant attention from researchers in the field of feature selection. These include particle swarm optimization (PSO) [17], artificial bee colony (ABC) [18], the genetic algorithm (GA) [19][20] and ant colony optimization (ACO) [21]. In recent years, numerous algorithms combining evolutionary optimization and reinforcement learning have been proposed. For example, [22] proposed an optimization algorithm that utilizes reinforcement learning to improve particle swarm optimization. In [23], a feature selection algorithm called QBSO-FS was proposed. This algorithm combines reinforcement learning and bee colony optimization to address the feature selection problem. Following the wrapper approach, to make the search process more adaptive and efficient, this study utilizes a combined BSO and Q-learning approach to generate feature subsets. These feature subsets are then evaluated using classifiers. In [24], the study discussed the role of the bee colony optimization algorithm combined with Q-learning optimization for feature selection in improving the prediction of heart disease.

3. Preliminaries

3.1. Feature selection

Let X be a fixed-dimensional input space, where k represents the number of features. Let T be the training set ($T \subseteq X$), $m \times k$ be dimensional, that is $T = \{u_i, y_i\}_{i=1}^m$, there are m training samples, and each sample consists of a k dimensional input vector $u_i = (u_i^1, u_i^2, \dots, u_i^k)$ (u_i from some fixed distribution above D_X) and a class label y_i . Let C represent the class label vector of this m training sample, i.e., $C = (y_1, y_2, \dots, y_m)$. For the binary classification problem, if there is a binary hypothesis $g : X \mapsto \{0, 1\}$, then $y_i = g(u_i) \in \{0, 1\}$. Let F represent the set of all k features, that is, $F = \{f_1, f_2, \dots, f_k\}$, F be a subset of F ($F \subseteq F$), and the training example $u_i|_F$ are limited to the feature subset F , that is, only the training sample obtained using the feature subset F . Similarly, the $X|_F$ representation input space is limited to a subset of features F . The goal of feature selection is to find a subset of features F , so that the hypothesis $h : X|_F \mapsto \{0, 1\}$ defined only on the feature subspace $X|_F$ can be generalized to the input space X (the generalization error of the hypothesis h is $\epsilon(h) = Pr_{u \in D_X}[h(u) \neq g(u)]$, and the empirical error on the training set T is $\hat{\epsilon}(h) = \frac{1}{|T|} \{(u, y) \in T | h(u) \neq y\}$) [25].

If hypothesis g uses all the features k , we expect to find the approximation function of the hypothesis g using only a small subset of the features k (according to the generalization error).

Let F be a finite set of features and terminates as an additional feature node f_s . The state space S of a Markov decision process (MDP) is a power set of features F , and the final state $F(\subseteq F)$ is all the included states f_s . The action space, similarly, is a collection of features. In any nonfinal state F , a possible action is to select a feature from F/F . Let $p : S \times F \times S \mapsto \mathbb{R}^+$ be the probability transfer function. If $F' = F \cup \{f\}$, $p(F, f, F')$ is nonzero.

Since the main goal of feature selection is to minimize the generalization error at the end of the learning process, the reward function associated with the final state F is naturally the generalization error of the learned hypothesis $\mathcal{A}(F/\{f_s\})$, denoted as $Err(\mathcal{A}(F))$. π is an MDP policy that performs an action in a certain state, F_π representing the final state (feature subset) of the policy π that starts with an empty state. Then, the optimal strategy π^* is obtained by minimizing the generalization error $Err(\mathcal{A}(F_\pi))$, i.e.,

$$\pi^* = \underset{\pi}{argmin} Err(\mathcal{A}(F_\pi)) \quad (1)$$

According to Bellman's optimal principle, the optimal value function V^* is defined as:

$$V^*(F) = \begin{cases} Err(\mathcal{A}(F)), & \text{if } F \text{ is final} \\ \min_{f \in F/F} V^*(F \cup \{f\}), & \text{otherwise} \end{cases} \quad (2)$$

The optimal policy π^* is defined as derived from V^* :

$$\pi^*(F) = \underset{f \in F/F}{argmin} V^*(F \cup \{f\}) \quad (3)$$

It can be proved that the above π^* is optimal. However, since the state space grows exponentially with the number of features, it is challenging to develop a solvable algorithm. The author of [11] provides an approximate solution for an optimal strategy that is based on the upper confidence tree (UCT) framework. This framework extends the trade-off between optimal exploration and utilization of the upper confidence bound (UCB) algorithm in sequence decision-making.

3.2. Q-learning

Q-learning is a model-free reinforcement learning algorithm used by agents to learn how to make optimal actions in a controlled Markov domain [26]. In Q-learning, an agent typically performs an action by transitioning between states in the environment and receives rewards or punishments based on the action taken to reach the target state [27]. Q-learning includes five basic components: agent, environment, action, state, and reward [28][29]. Consideration S refers to the range of potential states that a learning agent can be in within a given environment, and A represent the range of possible actions that the agent can choose and execute. Then, these components of agent i at moment t can be expressed as $s_t^i \in S$, $a_t^i \in A$, and $r_t^i \in S \times A$. Therefore, the total cumulative reward, that is, the Q-value. The updated formula is shown in Formula (4).

$$Q^i(s_t^i, a_t^i) \leftarrow Q^i(s_t^i, a_t^i) + \alpha[r_{t+1}^i + \gamma \max_a Q^i(s_{t+1}^i, a) - Q^i(s_t^i, a_t^i)] \quad (4)$$

Where $\alpha \in [0, 1]$ is the learning rate and $\gamma \in [0, 1]$ is the discount factor. The learning rate α can control the balance between exploration and exploitation. In other words, a smaller learning rate allows the algorithm to learn more about existing information, while a larger learning rate reduces the retention of previous Q-values.

3.3. Particle Swarm Optimization (PSO)

The conventional PSO is a bio-inspired evolutionary algorithm which is inspired by the food searching behaviors of fish or birds in nature. In this algorithm, each candidate solution to the optimization problem is represented by a particle, and each particle has two main properties that are position and velocity. Another important quantity in the PSO algorithm is the fitness function, which assesses the quality of each solution. Particles with higher fitness function values are considered to be of superior quality. Suppose the number of particles in the population is N , and the dimension of the feasible region space is D . The position and velocity of the particle i can be expressed as $x_i = (x_{i1}, x_{i2}, \dots, x_{iD})$ and $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$ respectively [17]. The particle updates the velocity and position of the next generation, respectively, according to equations (5) and (6).

$$x_{id}^{t+1} = x_{id}^t + v_{id}^{t+1} \quad (5)$$

$$v_{id}^{t+1} = w * v_{id}^t + c_1 * rand_1() * (pbest_{id}^t - x_{id}^t) + c_2 * rand_2() * (gbest_d^t - x_{id}^t) \quad (6)$$

where t denotes the t th iteration in the search process. $d \in D$ denotes the d th dimension in the search space. c_1 and c_2 are acceleration constants. $rand_1()$ and $rand_2()$ are random values uniformly distributed in $[0, 1]$. $pbest_{id}^t$ represents the d th dimensional position vector of the optimal solution position of particle i in t th iteration, $gbest_d^t$ represents the d th dimensional position vector of the optimal solution position of the particle swarm obtained empirically. w is inertia weight which ranges from 0 to 1.

The particle swarm optimization algorithm was initially introduced as an optimization technique in the real number space. However, many optimization problems occur in the discrete space. Kennedy and Eberhart [30] proposed a binary particle swarm optimization algorithm. The velocity update formula remains unchanged, and only the position is restricted to either 0 or 1.

4. EQL-FS

Among the metaheuristic optimization algorithms, the particle swarm optimization (PSO) algorithm has proven to be an effective method for solving optimization problems due to its robustness and ease of implementation. Compared to other optimization algorithms, PSO requires fewer parameters, is computationally less expensive, and can achieve convergence more rapidly [17]. Inspired by [2] and considering the advantages of reinforcement learning and evolutionary optimization algorithms, this paper utilizes Q-learning and PSO algorithms to select features. While Q-learning carries out feature selection, particle swarm optimization (PSO) can enhance the exploration capability of optimal strategies and achieve superior outcomes.

The Framework of EQL-FS. The single-agent method adopted by [2] has low exploration efficiency on high-dimensional feature data. The evolution Q-learning feature selection (EQL-FS) method proposed in this paper adopts multiagent method. Multiagent interaction is achieved using the PSO method. The idea of EQL-FS algorithm is as follows.

In each episode, all agents will first complete the Q-learning process, or the Q-table updating process, and then treat the Q-table obtained by each agent as a particle (the population number of particles equals the number of agents) and update it once through PSO evolution. The particle swarm optimization (PSO) algorithm returns the best agent among these agents and utilizes it to select features in a greedy manner, aiming to obtain the optimal feature subset during the current episodes. After all episodes have finished running, the feature subset with the highest accuracy in the test set of episodes will be checked first. If the test accuracy is the same, the feature selection strategy with the highest gain value will be selected by calculating the gain value (see Formula (7)). The architecture is shown in Fig. 1.

$$Gain = k_1 * Acc_Ts - k_2 * Len_FS / All_FS - k_3 * |Acc_Tr - Acc_Ts| \quad (7)$$

Where the coefficient $k_1, k_2, k_3 \in [0, 1]$, and $k_1 + k_2 + k_3 = 1$. Acc_Ts , Acc_Tr , len_FS , and All_FS represent the accuracy of the test set, the accuracy of the training set, the length of the chosen feature after the episodes end, and the total length of the feature in the dataset, respectively. Since it is necessary to consider the accuracy on the test set, the length of the selected features, and the difference in classification accuracy between the training set and the test set, Formula (7) is adopted.

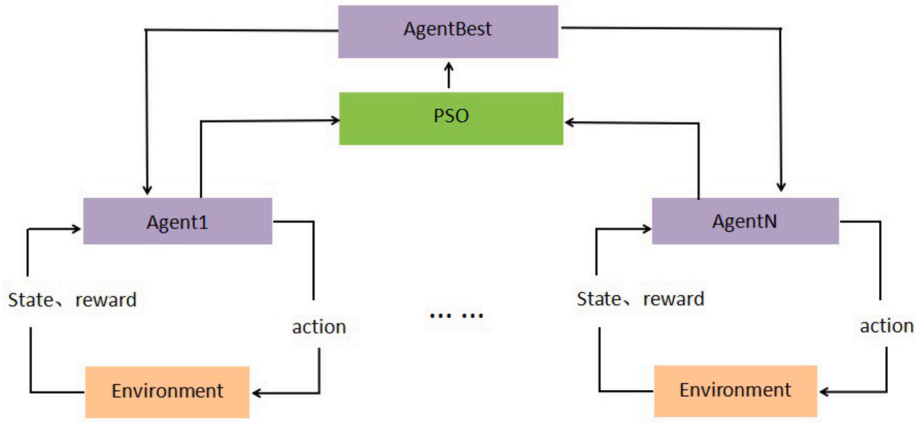


Fig. 1. EQL-FS architecture diagram.

The interaction of multiagent information occurs in the PSO algorithm (line 22 in the pseudocode, Algorithm 1) and in the weighting of the best agent (BestAgent) and the original agent (line 31, in Algorithm 1). The PSO algorithm adopted here is an algorithm that operates in the original continuous search space. Every time the PSO algorithm is triggered, each agent is inserted into the PSO algorithm as a particle for iterative updating, thereby obtaining the optimal agent, BestAgent. The Q-learning section (line 8-17 in the Algorithm 1) is similar to the code provided in the literature [2]. In this section, each agent begins in the initial state and uses the ϵ -greedy method to sequentially select features. The evaluate part (line 24-25 in the Algorithm 1) selects the optimal agent returned by the PSO algorithm. It adopts the greedy method to select the action (i.e., feature) that has the maximum action value (Q-value) in this state successively. Finally, it obtains the optimal feature subset.

Algorithm 1: EQL-FS algorithm.

```

Data: Train set; Test set
Result: Optimal feature subsets (Optimal policy);  $Acc\_Ts$ ;  $Acc\_Tr$ 
1 Parameter initialization;
2 error  $\leftarrow$  0;
3 last_error  $\leftarrow$  0;
4 for not at end of episodes_number do
5   Get episode_Data;
6   // Part of Train Datasets
7   for  $i \leftarrow 0$  to num_agents do
8     for not at End_state do
9       Sel_FS  $\leftarrow$  [];
10      // Abbreviations for Selected Features
11      next_state = current_action;
12      Sample_next_action ( $Q_i$ );
13      //  $\epsilon$ -greedy, Algorithm 2
14      Append Sel_FS  $\leftarrow$  next_action;
15      error  $\leftarrow$  DecisionTreeClassifier (episode_Data[Sel_FS]);
16      Reward  $\leftarrow$  last_error - error;
17      Update  $Q_i$ [current_state][current_action];
18      // Q_Learning
19      last_error  $\leftarrow$  error;
20    end
21  end
22   $Q\_best \leftarrow$  PSO( $Q_1, Q_2, \dots, Q_{num\_agents}$ );
23  for not at End_state do
24    next_state = current_action;
25    Sample_next_action( $Q\_best$ );
26    // Greedy
27  end
28   $Acc\_Tr \leftarrow$  DecisionTreeClassifier(Train set);
29   $Acc\_Ts \leftarrow$  DecisionTreeClassifier(Test set);
30  // return Optimal feature subsets;  $Acc\_Ts$ ;  $Acc\_Tr$ ;
31   $Q \leftarrow (1-\alpha) * Q\_best + \alpha * Q$ ;
32  //  $Q\_best$  inject into  $Q_i$ 
33 end

```

The Partial Parameter of EQL-FS. In EQL-FS, the state and action spaces correspond to the data feature space as in [2]. The current state reflects the selected feature from the previous state, and the current action involves selecting the next feature. In each

Algorithm 2: Sample_next_action algorithm.

```

input :  $Q\_Table$ 
output: action
1 if  $random() < \epsilon$  then
2   |  $action \leftarrow random\_choice(All\_FS - Sel\_FS)$ 
3 else
4   |  $action \leftarrow \operatorname{argmax}_{action \in \{All\_FS - Sel\_FS\}} Q[current\_state][action]$ 
5 end

```

Table 1
Datasets.

Datasets	# Features	# instances	# class	Source
segment	19	2308	2	KEEL
autos	25	159	6	KEEL
dermatology	34	358	2	KEEL
KDDcup_land_vs_satan	41	1610	2	KEEL
spectfheart	44	267	2	KEEL
spambase	57	4597	2	KEEL
sonar	60	208	2	KEEL
optdigits	64	5620	10	KEEL
ozone	72	2536	2	UCI
coil2000	85	9822	2	KEEL
movement_libras	90	360	15	UCI
musk	166	6598	2	UCI
Arrhythmia	279	452	16	UCI
DARWIN	450	174	2	UCI
isolet	617	7797	26	UCI
parkin	753	756	2	UCI

episode, the classification model outputs the prediction error at the current moment t . The reward is calculated as the prediction error from the previous moment minus the prediction error at the current moment. A greater prediction error at the previous moment compared to the current one results in a higher reward, while a minor prediction error leads to a reduced reward. Each agent updates the Q-table based on the rewards obtained and Formula (4), where the discount factor γ is set to 0.005. The remaining parameters, such as the learning rate α and ϵ , remain consistent with those in Table 1 of reference [2].

5. Experiment

5.1. Experiments of public datasets

5.1.1. Public datasets

This research collected sixteen public datasets to validate the superiority of the proposed algorithm. Details of the dataset are shown in Table 1 (DataSets). There are seven datasets from the UCI (<https://archive.ics.uci.edu/dataset>) and nine datasets from the KEEL (<http://www.keel.es/>). The datasets include segment, autos, dermatology, KDDcup_land_vs_satan (kdd), spectfheart, spambase, sonar, optdigits, ozone, coil2000, movement_libras, musk, Arrhythmia, DARWIN, isolet, and parkin.

5.1.2. Results

The classifier selects the decision tree classifier to obtain the accuracy on the corresponding dataset. The algorithms for comparison are CFS, FS-PSO, EQL-FS, ALL_F (using all features). The new algorithm, EQL-FS, integrates Q-learning and the particle swarm optimization algorithm. Therefore, CFS and FS-PSO are selected to compare and observe the performance when Q-learning or PSO is used alone.

CFS: curiosity feature selection method [2]. The CFS method is the benchmark algorithm in this paper. In the CFS method, the accuracy of the test set is used to compare with other feature selection algorithms in the selected dataset.

FS-PSO: discrete particle swarm optimization algorithm. The binary particle swarm optimization algorithm carries out Feature selection. In this paper, we use the FS-PSO, which represents a binary particle swarm optimization algorithm based on the original PSO algorithm, designed for discrete search space [30]. The velocity update formula remains unchanged, and only the position is restricted to either 0 or 1, according to a specified threshold set at 0.5 in this paper. When x_{id} exceeds this threshold, the position is encoded as 1. Otherwise, it is encoded as 0. Using its superior global search ability, the feature selection is performed by particle swarm optimization algorithm. Based on the existing literature research [31][32], we choose the parameters $c_1 = c_2 = 2$. The velocity v_{id}^t is limited by a predefined minimum and maximum velocity, $v_{id}^t \in [v_{min}, v_{max}]$, $v_{min} = -0.5$, $v_{max} = 0.5$. In addition, $w = 0.9$. The number of particles is 20, and the maximum number of iterations is 100. The feature selection problem of this paper needs to consider both the classification accuracy and the size of the feature subset. Therefore, the fitness function combines the accuracy and the proportion of features that are not selected as in [33]. This fitness function is defined as Formula (8).

Table 2

Accuracy and feature length of the optimal strategy on different datasets after averaging (DT) **The best results are shown in bold.

Datasets	Algorithms	Acc_test (%)	Slen_F (%)	Datasets	Algorithms	Acc_test (%)	Slen_F (%)
segment	EQL-FS	99.149	16.842	ozone	EQL-FS	98.431	3.056
	CFS	99.149	34.737		CFS	98.431	8.889
	FS-PSO	96.596	23.158		FS-PSO	96.863	47.5
	ALL_F	89.362	100		ALL_F	97.255	100
autos	EQL-FS	81.25	18.4	coil2000	EQL-FS	95	1.882
	CFS	78.75	28		CFS	94.919	1.176
	FS-PSO	63.75	24		FS-PSO	92.805	33.412
	ALL_F	62.5	100		ALL_F	89.106	100
dermatology	EQL-FS	100	5.294	movement_libras	EQL-FS	70	30.667
	CFS	100	4.706		CFS	68.889	74.667
	FS-PSO	100	9.412		FS-PSO	53.889	38.222
	ALL_F	100	100		ALL_F	59.444	100
kdd	EQL-FS	100	6.829	musk	EQL-FS	92.273	8.313
	CFS	100	9.756		CFS	91.818	99.036
	FS-PSO	95.758	9.756		FS-PSO	88.939	43.855
	ALL_F	99.394	100		ALL_F	89.394	100
spectfheart	EQL-FS	91.429	6.818	Arrhythmia	EQL-FS	77.391	47.455
	CFS	88.571	16.818		CFS	67.826	98.996
	FS-PSO	74.286	36.364		FS-PSO	65.217	46.667
	ALL_F	75.714	100		ALL_F	53.913	100
spambase	EQL-FS	91.478	84.211	DARWIN	EQL-FS	80	2.178
	CFS	91.391	85.614		CFS	75	99.511
	FS-PSO	88.348	49.123		FS-PSO	55	42.356
	ALL_F	89.391	100		ALL_F	65	100
sonar	EQL-FS	94.545	24.667	isolet	EQL-FS	76.667	40.032
	CFS	90.909	90.667		CFS	75.705	99.968
	FS-PSO	67.273	31.333		FS-PSO	72.179	50.373
	ALL_F	65.455	100		ALL_F	73.205	100
optdigits	EQL-FS	86.548	78.125	parkin	EQL-FS	83.684	15.485
	CFS	85.765	90		CFS	76.842	100
	FS-PSO	83.274	56.25		FS-PSO	68.947	48.287
	ALL_F	81.708	100		ALL_F	70.526	100

$$fitness = w' * Acc_Tr + (1 - w') * (1 - Len_FS) / All_FS \quad (8)$$

where Acc_Tr denotes the classification accuracy on the training set, and $(1 - Len_FS) / All_FS$ denotes the proportion of unselected features. Classification performance is key to feature selection. Compared with other optimization algorithm in our experiment, the PSO algorithm's feature selection length is relatively shorter (refer to Section 6.3). We choose the weight as $w' = 0.99$.

EQL-FS: This method is the new algorithm designed in this paper, i.e., the evolved Q-learning for feature selection optimization algorithm. The evolutionary optimization method adopts continuous particle swarm optimization because it optimizes the Q-value of multiple agents, which is a continuous real number. Therefore, the conventional particle swarm optimization algorithm is integrated with Q-learning in reinforcement learning to perform feature selection. In the conventional particle swarm optimization algorithm, the inertia weight is $w = 1$, $c_1 = c_2 = 2$, the velocity range is $[-1, 1]$, the population size is the number of agents, the maximum number of iterations is 50, and the fitness function is the same as that of the discrete particle swarm optimization algorithm. Since the solution now represents the Q-table, the particle swarm optimization algorithm is applied to search for the optimal solution for the agent. To prevent the Q-value from changing too much and reduce the difficulty of searching for the optimal solution, through the observation of the experimental process, the position x_{id} is restricted within $[-5, 5]$.

ALL_F: This method utilizes all the feature attributes of the dataset, selects the same classifier as other methods, and calculates the accuracy of the training and test sets without feature screening.

The performance of the four comparison methods is analyzed using various datasets. This analysis considers the accuracy of the test set and the proportion of the selected feature length to the total length. Table 2 displays the accuracy results of the four algorithms on various datasets. These results are obtained by averaging the optimal strategies from the five runs. The coefficients in Formula (7) are, $k_1 = 0.9$, and $k_2 = k_3 = (1 - k_1) / 2$, respectively.

Comparison of Accuracy. In all sixteen given datasets, with the exception of the four datasets (segment, dermatology, kdd, and ozone), the accuracy of EQL-FS is equal to that of CFS. However, in the remaining 12 datasets, the performance of EQL-FS is ranked first. If the ratio of the number of features used to achieve this accuracy is observed again with the same accuracy, only in the dermatology dataset can EQL-FS be regarded as the second-best choice among the four algorithms. In the segment, kdd and ozone datasets, although the EQL-FS algorithm and CFS can achieve the same accuracy, EQL-FS can achieve the same performance with a shorter subset of features. This shows that our new algorithm can accurately identify and remove redundant features without sacrificing accuracy performance. The FS-PSO algorithm selects the shortest number of features for the spambase, optdigits, and Arrhythmia datasets. However, the classification performance is not optimal. This shows that PSO may sacrifice accuracy to select fewer features, and it easily falls into local optima.

Notably, in the five datasets of musk, Arrhythmia, DARWIN, isolet and parkin, CFS did not find the feature subset with higher accuracy in the limited exploration step. Almost all features are used. The accuracy of ALL_F using all features is different from that

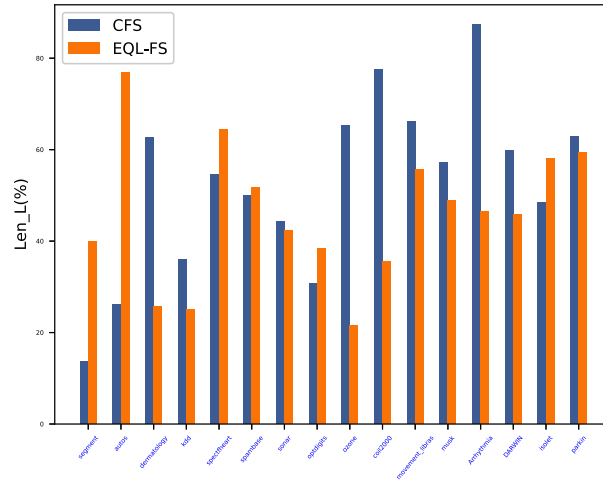


Fig. 2. Learning speed of the optimal strategy on different datasets after averaging (DT).

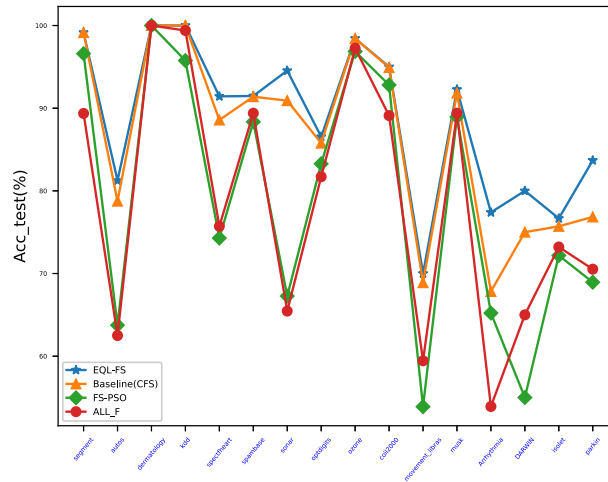


Fig. 3. Accuracy of the optimal strategy on different datasets after averaging (DT).

of CFS using a 100% feature set, mainly because in the feature selection process, the output accuracy of CFS is the result of one run. At the same time, ALL_F is the average of five runs. Alternatively, it could be due to random seeds. However, EQL-FS can explore better feature subsets, particularly on the musk and DARWIN datasets. Only 8.313% and 2.178% of the feature sets, respectively, can be used to achieve higher accuracy. This shows that PSO can improve the exploration efficiency of the agent when it is added to reinforcement learning for feature selection.

Comparison of Learning Speed. The length of each episode is set for each dataset. In an experiment, when the agent begins to select features, the agent begins to learn. The symbol “Len_L” represents the episode value divided by the corresponding length of the episodes, where the episode value is the length of beginning learning to find the optimal policy (highest classification accuracy or highest gain value). As seen from Fig. 2, on most datasets, the learning length needed for EQL-FS to learn the optimal policy is lower than that of CFS. According to the Q-value setting, CFS usually starts learning from the first episode, while EQL-FS may not start learning from the first episode due to the addition of the PSO algorithm. Meanwhile, the optimal agent obtained by the particle swarm optimization (PSO) in the previous episode is reassigned to each agent with weight again. Thus, the speed of optimization for EQL-FS is accelerated. Therefore, even on low-dimensional datasets, EQL-FS can not only find fewer features but also find them faster.

Based on the CFS algorithm, the EQL-FS algorithm can outperform or match the benchmark algorithm on a given dataset (refer to Fig. 3). Both the FS-PSO and ALL_F algorithms perform worse than the benchmark algorithm. However, on certain datasets, the FS-PSO algorithm outperforms the results obtained when using all features. The EQL-FS, CFS, and FS-PSO algorithms all possess the capability to eliminate noise features. However, the EQL-FS algorithm stands out for its superior ability to eliminate redundant features.

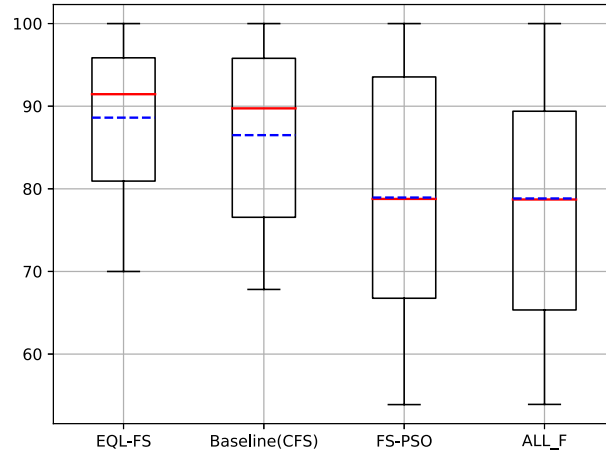


Fig. 4. Box diagram of the accuracy of the four algorithms after averaging the optimal strategy (DT).

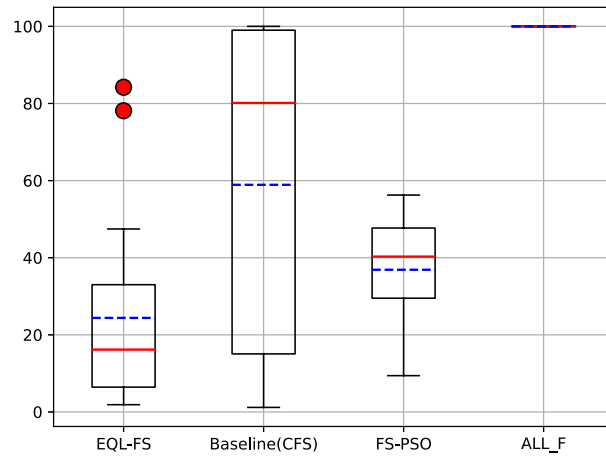


Fig. 5. Feature length boxplot of the four algorithms after averaging the optimal strategy (DT).

In general, if we prioritize classification performance in our practical problems, the EQL-FS algorithm can typically achieve superior classification results (refer to Fig. 4). If we focus on a smaller number of features, the EQL-FS algorithm is also more likely to find a subset of features that is shorter (Fig. 5). The two outliers in Fig. 5 are the spambase and optdigits datasets.

5.2. Experiments of a practical application dataset

Customers can choose services from various companies within the communications industry to fulfill their communication and internet requirements. Companies are evaluated by customers based on the quality of service they provide. To enhance customer experience and establish trust, companies may develop strategies aimed at reducing churn by improving service quality. Churn analysis is crucial for companies due to the high costs associated with acquiring new customers. In this subsection, we focus on a case study involving a company's monthly broadband customer base churn dataset for feature selection and assess the practical effectiveness of the proposed new algorithm. The features of the dataset are categorized into two types: categorical features and numerical features. Potential churn customers are identified based on both numerical features and categorical features. Numerical features have been binned and preprocessed for outliers, subsequently converted into categorical features. This process yields a total of fifty-seven features, which include the class label. To protect customer privacy, feature names are anonymized as "VARi". Given the large sample size of the dataset, we randomly select 50,000 samples for analysis.

We perform five rounds of experiments and present the experimental results under EQL-FS, CFS, and FS-PSO. These three algorithms are consistent with those described in Section 5.1.2. We select the optimal strategy for each round experiment. Table 3 displays the results.

Comparison of Accuracy. According to the results obtained under the optimal strategy, the average accuracy of EQL-FS on the test set is 99.416%. This performance surpasses that of both the CFS algorithm and the FS-PSO algorithm. Throughout five rounds of experiments, EQL-FS consistently demonstrated higher accuracy than CFS. It is also noteworthy that in one specific round of experiments, both EQL-FS and FS-PSO achieved the same accuracy of 99.36%. However, the number of features selected by the two

Table 3

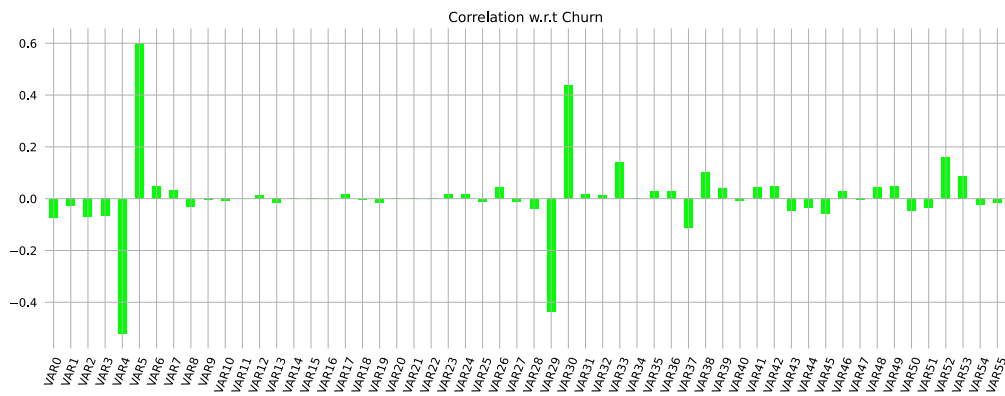
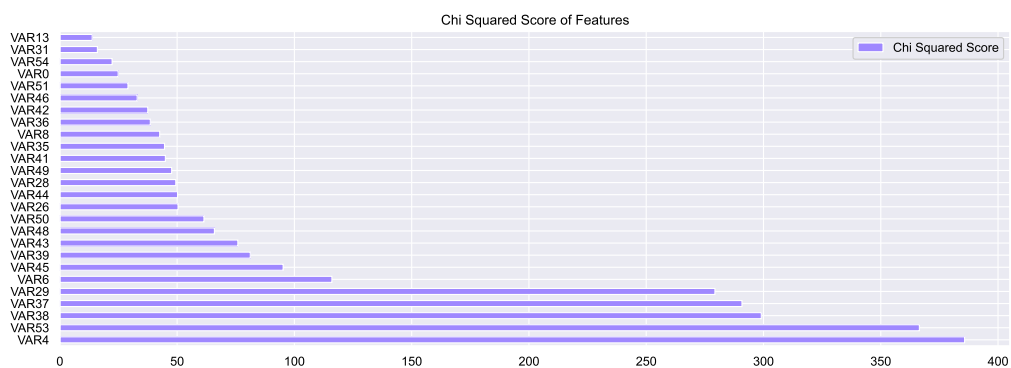
Accuracy and feature length of the optimal strategy on broadband customer base churn dataset (DT) **The best results are shown in bold.

	Acc_test (%)	Slen_F (%)		Acc_test (%)	Slen_F (%)		Acc_test (%)	Slen_F (%)
EQL-FS	99.36	19.642	CFS	99.24	69.643	FS-PSO	99.4	21.429
	99.44	25		99.24	48.214		99.32	30.357
	99.36	12.5		99.2	92.857		99.36	16.071
	99.48	48.214		99.24	51.786		99.36	25
	99.44	16.071		99.24	89.286		99.4	19.643
ave	99.416	24.286	ave	99.232	70.357	ave	99.368	22.5

Table 4

Comparison of features selected by EQL-FS and FS-PSO **The number “i” means feature “VARi”.

EQL-FS	Acc_test(%) = 99.36 Slen_F(%) = 12.5	2,5,23,30,37,38,46
FS-PSO	Acc_test(%) = 99.36 Slen_F(%) = 16.071	4,6,21,29,34,37,43,46,47
	Acc_test(%) = 99.36 Slen_F(%) = 25	4,6,15,25,27,29,30,35,36,37,39,42,54,55

**Fig. 6.** Correlation w.r.t. Churn.**Fig. 7.** Chi-squared test for all features.

algorithms differs. EQL-FS selects 12% of the total features, while FS-PSO selects either 16.071% or 25%. For a detailed comparison, refer to Table 4, which lists the features selected by both algorithms. Figs. 6 and 7 illustrate the correlation between each feature and the class label. After sorting the Chi-square test results in descending order, the top four features align perfectly with the findings presented in Fig. 6. Consequently, Fig. 7 includes only the features ranked from 5th to 30th based on their Chi-square test scores. In summary, as evidenced by Table 4, Fig. 6, and Fig. 7, EQL-FS can identify features that exhibit a stronger correlation with the ground truth.

Table 5

Comparison of classification performance between EQL-FS and CFS under selected features **The number “i” means feature “VARi”. The F1, Recall, and Gmean are machine learning evaluation metrics realized by the Sklearn Codebase.

		The Selected Features	F1	Recall	Gmean
EQL-FS	Acc_test(%) = 99.48 Slen_F(%) = 48.214	0,2,4,6,15,16,18,20,21,22,24,26,29,30,31,32,34,36,37,40,41,46,48,49,52,53,55	0.813	0.744	0.862
CFS	Acc_test(%) = 99.24 Slen_F(%) = 48.214	5,8,9,10,11,14,15,16,17,18,19,20,21,25,27,30,36,38,40,41,44,45,46,47,50,52,55	0.739	0.718	0.846
ALL_F			0.698	0.729	0.851

Comparison of the Selected Features. A slight difference exists between the lengths of features selected by the EQL-FS and FS-PSO algorithms. Compared to the CFS algorithm, the number of features selected by the EQL-FS algorithm is only about one-third of the CFS algorithm on average. In a particular round of experiments, EQL-FS and CFS select the same number of features.

Due to the imbalanced sample sizes of the two classes in the broadband customer base churn dataset, Table 5 presents additional results based on various evaluation metrics. As shown in Table 5, when selecting an equal number of features, EQL-FS identifies features that enhance classification performance and provide greater reference value for predicting potential customer churn.

6. Other related experiments

From the analysis of the experimental results above, it is evident that the newly designed algorithm demonstrates superiority in terms of accuracy, feature length needed to achieve optimal accuracy, and learning duration to acquire the optimal strategy. First, the classifier used in this paper is the same as the classifier used in the benchmark algorithm. If other classifiers are replaced simultaneously, will the new algorithm still outperform the benchmark algorithm? Second, in terms of the number of agents, the benchmark algorithm (CFS) adopts a single-agent approach, while our new algorithm (EQL-FS) adopts a multiagent approach. Can the multiagent approach achieve the same level of performance as the new algorithm we proposed? Thirdly, numerous evolutionary optimization algorithms have been developed to date. Why not consider alternative optimization algorithms? Finally, considering that the benchmark algorithm adopts a curiosity-driven feature selection method with a threshold hyperparameter setting, can different thresholds achieve the same performance comparison? To address the four questions raised here, this section conducts stability experiments using various classifiers, explores different multiagent algorithms, examines a range of optimization algorithms, and performs sensitivity analyses with varying parameters.

6.1. Stability experiments of different classifiers

Now, the classification results of the KNN classifier are described. According to the average accuracy ranking on the test set in Table 6, the EQL-FS algorithm remains in first place, followed by CFS, FS-PSO, and ALL_F. Even with different classifiers, EQL-FS can still obtain better strategies, indicating that the new algorithm is stable across different classifiers.

In autos, movement_libras, and DARWIN, there is a significant difference in accuracy among the four algorithms or a low level of accuracy. In fact, based on the characteristics of these three datasets, it can be observed that the sample size is small. This can result in overfitting, leading to low accuracy on their test sets and making it difficult to achieve satisfactory classification accuracy.

6.2. Experiments of different multiagent algorithms

Considering the influence of multiple agents, we chose the multiagent algorithm (CMRL [14]) and its variant CMRL2 for performance comparison with EQL-FS.

CMRL (coordination multiagent feature selection) [14]: The core idea of this algorithm is to “assign” a learning agent to each feature. Thus, the total number of learning agents is equal to the number of features. Each reinforcement learning agent can control a single feature, meaning that each agent determines whether a feature is included in the final feature subset. Each agent has two actions: “0” and “1”. Action “0” means that the feature will not be included in the feature subset. Similarly, the action “1” indicates that the feature will be included in the feature subset. Therefore, all actions labeled with a “1” will be included in our feature subset. Since the algorithm is a multiagent cooperative learning method, the information interaction between agents is determined by the global reward (see Formula (9)). The global reward is calculated as the reward obtained from the unselected features in an episode minus the reward obtained from the selected features. This reward is then used to update the Q-value of the opposite actions for each agent (e.g., in the same episode, the action made by agent i corresponding to feature i is “1”, and the opposite action is “0”. If the action made by agent i corresponding to the feature i is “0”, then the opposite action is “1” is updated (see Formula (10)), allowing for the exchange of information between agents.

$$R_{Global} = R(Unselected_FS) - R(selected_FS) \quad (9)$$

$$Q_i[1 - a_i] = Q_i[1 - a_i] + \alpha(R_{Global} - Q_i[1 - a_i]) \quad (10)$$

Table 6

Accuracy and feature length of the optimal strategy on different datasets after averaging (KNN) **The best results are shown in bold.

Datasets	Algorithms	Acc_test (%)	Slen_F (%)	Datasets	Algorithms	Acc_test (%)	Slen_F (%)
segment	EQL-FS	95.745	36.842	ozone	EQL-FS	100	18.056
	CFS	97.872	68.421		CFS	100	70.833
	FS-PSO	91.489	42.105		FS-PSO	98.039	41.667
	ALL_F	93.617	100		ALL_F	98.039	100
autos	EQL-FS	43.75	12	coil2000	EQL-FS	95.122	1.177
	CFS	50	16		CFS	94.919	1.177
	FS-PSO	31.25	32		FS-PSO	94.106	41.177
	ALL_F	12.5	100		ALL_F	94.715	100
dermatology	EQL-FS	100	2.941	movement_libras	EQL-FS	25	16.667
	CFS	100	2.941		CFS	27.778	82.222
	FS-PSO	100	2.941		FS-PSO	22.222	50
	ALL_F	100	100		ALL_F	16.667	100
kdd	EQL-FS	93.939	2.439	musk	EQL-FS	93.939	14.458
	CFS	93.939	2.439		CFS	90.152	93.976
	FS-PSO	93.939	7.317		FS-PSO	89.394	39.157
	ALL_F	93.939	100		ALL_F	89.394	100
spectfheart	EQL-FS	100	52.273	Arrhythmia	EQL-FS	69.565	29.032
	CFS	92.867	52.273		CFS	65.217	97.85
	FS-PSO	85.714	43.182		FS-PSO	60.87	41.219
	ALL_F	85.714	100		ALL_F	65.217	100
spambase	EQL-FS	83.913	17.544	DARWIN	EQL-FS	50	48.222
	CFS	79.13	17.544		CFS	25	98.889
	FS-PSO	83.913	35.088		FS-PSO	50	39.556
	ALL_F	74.348	100		ALL_F	25	100
sonar	EQL-FS	90.909	18.333	isolet	EQL-FS	50	58.671
	CFS	90.909	15		CFS	48.718	99.838
	FS-PSO	54.546	33.333		FS-PSO	53.526	50.891
	ALL_F	81.818	100		ALL_F	48.397	100
optdigits	EQL-FS	86.833	87.5	parkin	EQL-FS	73.684	0.133
	CFS	86.833	89.063		CFS	65.79	99.867
	FS-PSO	85.053	57.813		FS-PSO	71.053	43.559
	ALL_F	85.053	100		ALL_F	65.79	100

CMRL2: This algorithm is a variant of the CMRL method. It still follows the same agent design approach as the CMRL algorithm, where each agent corresponds to a feature. The information interaction between the agents is obtained from the global reward. Since the Q-value update mode of CMRL is slightly different from that of standard Q-learning, and both the new algorithm designed in this paper and the comparison algorithm CFS adopt the Q-value update mode of standard Q-learning, CMRL2 differs from CMRL in the following aspects. In CMRL2, more emphasis is placed on the optimization goal of Q-learning, which is to maximize the cumulative reward. We consider the global reward as a cumulative reward. The state space and action space for each agent are the same. The next state for each agent is determined by the last action taken. Each agent has its own Q-table, and the Q-values in the table are updated using the standard Q-learning update mode (refer to Formula (11)). The variant of CMRL is called CMRL2. (The implementation code for CMRL refers to the code on github: <https://github.com/hahajain/Feature-Selection-using-Reinforcement-Learning>)

$$Q_i[s_i][a_i] = Q_i[s_i][a_i] + \alpha((-R_{Global}) + \gamma \max_{a_k \in \{0,1\}} Q_i[s_{i+1}][a_k] - Q_i[s_i][a_i]) \quad (11)$$

For illustration, we selected the following datasets, which cover different feature lengths: autos, dermatology, kdd, spambase, optdigits, coil2000, Arrhythmia, DARWIN, and isolet.

As seen from the performance of the nine previously mentioned datasets (see Table 7), the average rank of EQL-FS accuracy remains first, followed by CMRL2 in second place. Both CFS and CMRL rank third. This result may be because EQL-FS incorporates multiple agents and particle swarm optimization algorithms, while CMRL2 and CMRL are only multiagent methods. This suggests that the number of multiagents does have an impact on classification performance and the selection of feature subsets. However, increasing the number of agents only limits their exploration ability. The ranking of CMRL2 is closer to that of our new algorithm, indicating that the agent's reward design is also an influential factor that cannot be ruled out. Another issue worth noting is that the number of agents in CMRL and CMRL2 is equal to the length of the original dataset's features. The number of agents is fixed and cannot be changed, but it increases as the number of features increases. The number of agents in the proposed algorithm is not fixed and cannot be changed. It can be much smaller than the number of agents in CMRL and CMRL2 (the number of agents in this paper is three). Based on the characteristics of the dataset, the number of agents can be selected flexibly.

6.3. Experiments of different optimization algorithms

This section examines the training accuracy of the returned solutions to illustrate the applicability of the particle swarm optimization algorithm. Numerous optimization algorithms have been proposed, and we have randomly selected three for comparison. GWO, the gray wolf optimization algorithm, is a new evolutionary computation technique that mimics gray wolves' leadership hierarchy

Table 7

Results of different multiagent algorithms **The best results are shown in bold.

Datasets	Algorithms	Acc_test (%)	Sen_F (%)	Datasets	Algorithms	Acc_test (%)	Sen_F (%)
autos	EQL-FS	81.25	18.4	coil2000	EQL-FS	95	1.882
	CFS	78.75	28		CFS	94.919	1.176
	CMRL	77.5	13.6		CMRL	95	8.941
	CMRL2	83.75	39.2		CMRL2	93.902	13.176
dermatology	EQL-FS	100	5.294	Arrhythmia	EQL-FS	77.391	47.455
	CFS	100	4.706		CFS	67.826	98.996
	CMRL	100	5.294		CMRL	77.391	31.398
	CMRL2	100	11.765		CMRL2	79.13	46.093
kdd	EQL-FS	100	6.829	DARWIN	EQL-FS	80	2.718
	CFS	100	9.756		CFS	75	99.511
	CMRL	100	3.902		CMRL	75	12.089
	CMRL2	100	8.293		CMRL2	75	32.533
spambase	EQL-FS	91.478	84.211	isolet	EQL-FS	76.667	40.032
	CFS	91.391	85.614		CFS	75.705	99.968
	CMRL	89.565	48.07		CMRL	75.833	31.864
	CMRL2	90.87	42.807		CMRL2	77.436	49.238
optdigits	EQL-FS	86.548	78.125				
	CFS	85.765	90				
	CMRL	84.27	88.438				
	CMRL2	85.907	47.5				

Table 8

Classification accuracy of different optimization algorithms (DT) **The best results are shown in bold.

segment	spectfheart	spambase	optdigits	ozone	coil2000	musk	parkin	ave
FS-PSO	95.563 ± 2.079	72.881 ± 9.238	88.267 ± 1.546	96.455 ± 1.592	92.08 ± 0.996	88.918 ± 2.312	72.292 ± 4.769	86.186 ± 3.042
GWO	93.608 ± 3.670	73.714 ± 6.197	87.391 ± 1.520	80.631 ± 2.393	96.767 ± 1.098	89.069 ± 0.752	88.844 ± 1.810	72.278 ± 4.484
WOA	92.669 ± 2.892	73.675 ± 6.574	86.864 ± 2.131	80.985 ± 2.946	96.610 ± 1.394	89.369 ± 1.134	88.959 ± 1.914	71.257 ± 4.350
BBA	93.648 ± 3.798	71.960 ± 9.295	86.068 ± 2.097	78.565 ± 3.377	96.041 ± 1.850	89.494 ± 0.678	88.729 ± 2.074	73.021 ± 3.980

Table 9

Feature proportions selected under different optimization algorithms (%) **The best results are shown in bold.

segment	spectfheart	spambase	optdigits	ozone	coil2000	musk	parkin	ave
FS-PSO	22.281 ± 5.474	32.803 ± 6.251	52.281 ± 6.793	54.115 ± 4.741	44.861 ± 5.557	35.137 ± 6.322	46.888 ± 2.657	47.747 ± 2.044
GWO	59.474 ± 11.070	56.970 ± 9.264	63.275 ± 6.925	62.084 ± 5.744	63.519 ± 5.638	66.118 ± 6.513	65.281 ± 4.794	64.387 ± 1.777
WOA	56.316 ± 15.949	62.576 ± 16.364	59.240 ± 11.093	64.219 ± 10.890	59.815 ± 12.217	58.902 ± 11.898	60.200 ± 8.773	60.545 ± 10.455
BBA	37.018 ± 8.902	45.000 ± 9.312	52.281 ± 8.375	50.261 ± 8.450	49.722 ± 9.252	52.157 ± 6.327	54.739 ± 7.586	51.957 ± 7.703

and hunting mechanism in nature [34]. WOA, the whale optimization algorithm, belongs to the family of stochastic population-based algorithms. It mimics the bubble-net feeding in the foraging behavior of humpback whales [35]. BBA is a binary version of the bat algorithm, and the bat algorithm (BA) is one of the recently proposed heuristic algorithms imitating the echolocation behavior of bats to perform global optimization [36]. These three algorithms are implemented using Py.fs [37], and the parameters are all set by default. The parameters of the FS-PSO algorithm are consistent with those of Section 5.1.2. The four algorithms' population size and number of iterations are also consistent: the population size is 20, and the number of iterations is 100. We selected eight datasets from the public dataset described in Section 5.1.1, which varied in sample sizes and feature counts. The classifier employed for this analysis is decision trees. Moreover, each algorithm is independently run 30 times to avoid random bias, and the average results are presented. Table 8 presents the mean and standard deviation of the training accuracy obtained from four optimization algorithms. Table 9 presents the average value and standard deviation of the percentage of the number of features selected by the optimization algorithm to the total number of features in each dataset.

We can observe from Table 8 that the FS-PSO algorithm achieves relatively high classification accuracy on half of the datasets. The GWO algorithm attained good accuracy on the spectfheart and ozone datasets, while the WOA and BBA algorithms performed well on one dataset each. However, based on the average results across the eight datasets, the overall classification performance of FS-PSO remains the best. Furthermore, the experimental results indicate that the average standard deviation of the classification accuracy of these four optimization algorithms across the eight datasets is approximately 3, suggesting that the degree of fluctuation is relatively consistent.

Table 9 presents the proportion of selected features for four optimization algorithms across eight datasets. Except for the optdigits dataset, which ranks second in the proportion of selected feature lengths, FS-PSO still achieves the fewest features in the remaining datasets. Furthermore, the degree of variation in the proportion of feature selection is relatively low, indicating that the random initial value of FS-PSO has minimal impact on the stability of the results.

6.4. Sensitivity experiments with different parameters

6.4.1. The size of the threshold

Different thresholds affect both the optimal policy and the average policy. After much debugging, we have observed that when we incorporate PSO optimization into reinforcement learning, the action value tends to increase slightly. As a result, the threshold is

Table 10
Results of different thresholds on partial datasets.

Datasets	Algorithms	Threshold	Acc_train (%)	Acc_test (%)	Slen_F (%)
segment	EQL-FS	2.5	96.635	100	15.789
		2.6	98.798	100	52.632
	CFS	0.01	99.279	97.872	21.053
		0.05	99.279	100	31.579
autos	EQL-FS	2.5	93.75	81.25	12
		2	75	81.25	28
		2.3	75	75	8
	CFS	0.3	81.25	75	12
		0.4	81.25	81.25	28
dermatology	EQL-FS	2.5	96.923	100	2.941
		2.6	98.462	100	2.941
	CFS	0.02	100	100	8.824
		0.05	100	100	14.706
kdd	EQL-FS	2.5	100	100	2.439
		2	100	100	19.512
		2.4	100	100	4.878
	CFS	0.05	100	100	12.195
		0.02	100	100	7.317
ozone	EQL-FS	2.5	97.155	98.039	1.389
		2.7	94.967	100	9.722
		2.8	97.155	98.039	1.389
	CFS	0.05	94.748	98.039	15.278
		0.005	96.28	98.039	8.333
coil2000	EQL-FS	2.4	93.691	95.122	2.353
		2.5	93.623	94.919	1.176
	CFS	0.03	93.623	94.919	1.176
		0.05	93.623	94.919	1.176
musk	EQL-FS	2.8	93.35	91.667	1.205
		<i>in</i> = 2.8 <i>ex</i> = 2.9	94.865	91.667	16.265
		<i>in</i> = 2.7 <i>ex</i> = 2.9	95.034	90.152	9.036
		0.05	94.781	92.424	97.59
	CFS	0.9	95.034	90.909	19.88
parkin	EQL-FS	<i>in</i> = 2.8 <i>ex</i> = 3.5	84.211	84.211	13.015
		<i>in</i> = 2.8 <i>ex</i> = 3.6	76.316	81.579	14.21
		<i>in</i> = 2.7 <i>ex</i> = 3.2	76.316	78.947	18.725
		0.01	77.193	78.947	100
	CFS	<i>in</i> = 0.99 <i>ex</i> = 0.999	74.561	76.316	100

larger compared to that in the CFS algorithm. The internal and external thresholds of the EQL-FS algorithm are approximately above 2.5, while the internal and external thresholds of the original CFS algorithm are roughly between 0 and 1.0 to determine the optimal policy. Table 10 displays the performance of the most recent optimal policy with various thresholds in several datasets. In this table, “in” denotes the internal threshold, “ex” represents the external threshold, and the bold mark indicates the final threshold chosen in this study. If the threshold value is not specified, the internal and external thresholds are the same. Since the Q-learning in this paper utilizes internal and external thresholds for feature selection, the values of the internal and external thresholds are experimented with, while keeping the parameters of other algorithms the same as those in the original literature [2].

6.4.2. The episode size

Since we are using a wrapper method for feature selection, the optimal feature subset is selected based on the feedback from the classifier’s performance. If the length of the feature set is k , then the size of the power set of the feature set (the set of all subsets of the feature set) is 2^k . As mentioned above, among the five datasets with high dimensions (musk, Arrhythmia, DARWIN, isolet,

Table 11

Pairing test of four algorithms (DT).

Alternative	EQL-FS > CFS	CFS>FS-PSO	CFS>ALL_F
pvalue	0.001109	0.000327	0.000327

Table 12

Pairing test of four algorithms (KNN).

Alternative	EQL-FS > CFS	CFS>FS-PSO	CFS>ALL_F
pvalue	0.054755	0.122746	0.001672

Table 13

Pairing test for feature length selection by three algorithms (KNN).

Alternative	EQL-FS<CFS	FS-PSO<CFS	EQL-FS<FS-PSO
pvalue	0.002924	0.017800	0.020444

parkin), the exploration performance of CFS is inferior to that achieved by the new algorithm. Therefore, a natural question is, will CFS eventually be able to explore the optimal strategy when our episode length is long enough? And outperform our new algorithm? Here, we conducted some experiments. In the Arrhythmia dataset, for instance, when using the KNN classifier, the episode length of the CFS algorithm has only been increased by 1.7 times compared to the original CFS algorithm, while keeping other parameters constant. The result of the optimal strategy shows that the accuracy of both the training set and the test set has not improved. The length of the selected feature subset is reduced from 97.85% to 72.76%, which is still not better than the result of the new algorithm EQL-FS. The parkin dataset is another example, with a total feature length of 753. When the episode length is increased to 2.8 times the original length, the shorter feature subset still cannot be found under the DT classifier, and the accuracy remains unchanged. The results show that the CFS algorithm still selects all features as the optimal strategy. Another example is the sonar dataset under the DT classifier. When the episode length of CMRL and CMRL2 is increased to 8.7 times the original length, the performance does not show significant improvement.

It can be observed that acquiring the optimal strategy does not primarily depend on the length of the episode. It is not necessarily needed to increase the length of the episode to obtain the optimal strategy. The discovery of the optimal strategy or optimal feature subset mainly relies on the algorithm's exploration and exploitation abilities. Therefore, in comparison, our proposed new algorithm can achieve better results with a limited episode length, and its exploration capability shows promise.

7. Signed rank test

The statistical test results of several experiments are given in this section. In Table 2, the data show the average accuracy of four algorithms: EQL-FS, CFS, FS-PSO and ALL_F. The EQL-FS algorithm ranks first, CFS ranks second, ALL_F ranks third, and FS-PSO ranks fourth. First, the Friedman test is used to determine whether there are significant differences between the four algorithms at a specific significance level (in this paper, the significance level $\alpha = 0.05$). When there are significant differences between the four algorithms, the Wilcoxon signed-rank test is used to conduct paired test of two algorithms.

Through calculation, the statistic of Friedman test is 39.857 (statistic = 39.857), and the p value is close to zero ($pvalue = 1.142 \times 10^{-8}$). The p value is less than 0.05, indicating that there are significant differences among the four algorithms in Table 2.

Based on the CFS method, the pairwise matching test of the pairwise algorithm was conducted, and the results are presented in Table 11. The vertical axis represents the alternative hypothesis, while the horizontal axis represents p value. Under the condition of a significance level $\alpha = 0.05$, the new algorithm EQL-FS is better than the CFS algorithm, but the accuracy of FS-PSO with all features is lower than that of the CFS algorithm.

Based on KNN classifier, we also utilized Friedman test and Wilcoxon signed-rank test to conduct the accuracy performance evaluation. The statistic of the Friedman test is 22.382, $pvalue = 5.43 \times 10^{-5}$, and the four algorithms show significant differences in statistical significance. The remaining results are shown in Table 12.

As shown in Table 12, there is no significant difference between the EQL-FS algorithm and CFS algorithm. Similarly, there is no significant difference between the CFS algorithm and the FS-PSO algorithm. However, when considering the features selected to obtain the corresponding accuracy rate, there is a difference in feature length, as indicated in Table 13.

In the sixteen public datasets we selected, the average rank of EQL-FS is still higher than that of the other three algorithms when using the KNN classifier. It can be seen from Table 13 that the length of the feature subset selected by the EQL-FS algorithm is significantly lower than that of FS-PSO and CFS. Therefore, whether based on the DT or KNN classifier, the performance of the new algorithm EQL-FS is significantly better than that of the other three algorithms.

8. Conclusion and discussion

Conclusion. Based on the literature [2], this paper proposes a new feature selection algorithm called EQL-FS. The algorithm combines reinforcement learning and evolutionary optimization techniques, utilizing Q-learning for reinforcement learning and PSO for evolutionary optimization. Therefore, this algorithm is referred to as evolutionary Q-learning feature selection (EQL-FS). Since it is difficult for CFS to explore the optimal strategy in a limited number of exploration steps when the feature dimension is relatively

high and the evolutionary optimization algorithm has strong exploration and optimization abilities, the particle swarm optimization algorithm is integrated with CFS to enhance the optimal subset searching ability of reinforcement learning methods in feature selection. To compare the two algorithms, we selected the CFS, FS-PSO, CMRL, CMRL2, and ALL_F methods for comparison. FS-PSO adopts the discrete particle swarm optimization algorithm, CMRL, and CMRL2 multiagent feature selection algorithms. On the other hand, ALL_F utilizes all feature attributes to improve classification accuracy. We utilize sixteen public datasets along with a broadband customer base churn dataset to validate the effectiveness of our proposed algorithm. The experimental results show that (1) EQL-FS, based on decision tree classifiers, consistently achieves higher accuracy and selects the smallest number of feature subsets. The results have statistical significance at a certain significance level. (2) Since the wrapper method relies on the classification performance of the classifier as a reward incentive, it is possible for different classifiers to yield varying classification accuracies. Therefore, in the following paper, the decision tree (DT) classifier is replaced with the k-nearest neighbors (KNN) classifier, and the algorithm's performance is compared on various datasets. Based on the KNN classifier, the average accuracy rank of EQL-FS is still higher than that of the other three algorithms (CFS, FS-PSO, ALL_F). Furthermore, the conclusion that EQL-FS selects the shortest feature subset remains statistically significant. The effect of the new algorithm is similar to that of the DT classifier, and in high dimensions ($d > 100$), better exploration results can still be obtained on five datasets of 100. This once again proves our assumption that the efficiency of exploring feature subsets is improved when we add the PSO method. (3) The increase in the number of reinforcement learning agents alone may still not lead to optimal results. Through the analysis of the nine selected datasets, it is evident that the multiagent algorithms CMRL and CMRL2 are unable to outperform the EQL-FS algorithm. In terms of the number of agents, both CMRL and CMRL2 have the same number of agents as the number of features in a given dataset. This number is fixed and will increase with the dimension of the dataset. However, EQL-FS allows for flexible design of the number of agents. In terms of agent rewards, the reward design of CMRL2 is more similar to our new algorithm. This could be one reason why the two algorithms are closely ranked. (4) The feasibility of choosing the particle swarm optimization algorithm. The four optimization algorithms FS-PSO, GWO, WOA, and BBA are compared across eight selected datasets. The experimental results further confirm the viability of utilizing the particle swarm optimization algorithm discussed in this paper. (5) Influence of Hyperparameters. The paper involves the setting of certain hyperparameters. In addition to being consistent with the parameters in the original literature [2], it is found that the threshold and episode length are the most influential factors in the experiment. The paper analyzes the influence of different hyperparameter values on the results. (6) Pairwise test. In the end, the Friedman test and Wilcoxon paired test further verify the conclusion described in our previous experimental results. Specifically, our proposed algorithm can improve the utilization rate of feature attributes without sacrificing classification accuracy. Additionally, it achieves better classification performance by using the shortest feature subset. Even when using different classifiers, all the proposed algorithms yield the same argument results, demonstrating the robustness of the new algorithm across various classifiers.

Discussion. Reinforcement learning can facilitate automatic feature selection. The application of evolutionary optimization algorithms to feature selection is primarily attributed to their superior global search capabilities. In a typical reinforcement learning algorithm, a single agent learns through multiple episodes to achieve the goal. If a learning problem is complex, it may require a significant amount of computational time to determine the optimal strategy. The combination of reinforcement learning and evolutionary optimization algorithms has also been explored for a long time [38]. In 2006, [39] proposed a multiagent based reinforcement learning algorithm. In this algorithm, multiple agents learn not only through their own experiences but also through interactions with each other. The agents update all Q-values based on the evaluation results of Q-values and their interactions with each other. The effectiveness of the algorithm is tested in the shortest path problem. In recent years, there has been a proliferation of proposals for hybrid algorithms known as evolutionary reinforcement learning (ERL). In [40], the authors utilized an evolutionary algorithm (EA) population to generate diverse data for training reinforcement learning agents. They also periodically incorporated intelligent weights into the evolutionary population to introduce gradient information into the evolutionary algorithm. However, these methods focus on optimizing the parameters of the policy network, which is often high-dimensional and challenging for evolutionary algorithms. To reduce the dimensionality of the evolutionary algorithm, [41] considers shifting the evolutionary goal from the high-dimensional parameter space to the low-dimensional action space. They propose the evolutionary action selection-double delay deep deterministic strategy gradient (EAS-TD3), which is a novel hybrid algorithm that combines an evolutionary algorithm (EA) and deep reinforcement learning (DRL). In the context of evolutionary action selection, the authors concentrate on optimizing the actions chosen by the strategy network. They aim to obtain high-quality actions by utilizing evolutionary algorithms, which in turn facilitate strategy learning. These hybrid algorithms, which combine reinforcement learning and evolutionary optimization, are primarily designed for continuous motion tasks, such as those in MuJoCo.

The combination of reinforcement learning and evolutionary optimization algorithms has shown advantages in the previous description. To enhance the classification performance of high-dimensional datasets, the utilization of reinforcement learning algorithms for feature selection has emerged as a relatively new approach.

9. Future work

The methods mainly involved in this paper include reinforcement learning methods and evolutionary optimization algorithms. In future research, we will focus on improving and discussing the following points: (1) Currently, most datasets have a high feature attribute dimension, whereas the feature dimension of the datasets used for comparative analysis in this paper is not excessively high. Therefore, when faced with high-dimensional data, we will continue to choose datasets with increasing dimensions to evaluate the algorithm's exploratory capabilities. (2) This paper utilizes the Q-learning method for reinforcement learning and the particle swarm optimization (PSO) algorithm for evolutionary optimization. In this paper, Q-learning employs multiple agents (specifically, 3 agents),

and the population iteration component of PSO also introduces complexity to the algorithm. Therefore, future research will focus on reducing the algorithm's complexity. (3) This paper only presents the feature subset, but it does not explain the features included in the subset. We will combine the filter method with additional explanations of each feature to better understand its importance and explainability. (4) In this paper, the fusion of reinforcement learning methods and evolutionary optimization algorithms is relatively straightforward. We will attempt to integrate reinforcement learning with other evolutionary optimization algorithms, merge deep reinforcement learning with PSO, and combine deep reinforcement learning with other evolutionary optimization algorithms.

CRedit authorship contribution statement

Guan Yang: Writing – review & editing, Writing – original draft, Software, Methodology, Data curation. **Zhiyong Zeng:** Software, Funding acquisition. **Xinrui Pu:** Writing – review & editing. **Ren Duan:** Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors would like to thank our team members for helpful discussions on topics related to this work. We would like to thank the anonymous reviewers for their helpful remarks, and the editor for their useful feedback that improved this paper. This research was partially supported by National Natural Science Foundation of China (NSFC) under grant No. 62166045.

Appendix A. Supplementary material

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.ins.2025.122441>.

Data availability

Data will be made available on request.

References

- [1] V. Kumar, S. Minz, Feature selection: a literature review, *Smart Comput. Rev.* 4 (2014) 211–229, <https://doi.org/10.6029/smarterc.2014.03.007>.
- [2] M. Moran, G. Gordon, Curious feature selection, *Inf. Sci.* 485 (2019) 42–54, <https://doi.org/10.1016/j.ins.2019.02.009>.
- [3] A. Blum, P. Langley, Selection of relevant features and examples in machine learning, *Artif. Intell.* 97 (1997) 245–271, [https://doi.org/10.1016/S0004-3702\(97\)00063-5](https://doi.org/10.1016/S0004-3702(97)00063-5).
- [4] M. Robnik-Šikonja, I. Kononenko, Theoretical and empirical analysis of relieff and rrelieff, *Mach. Learn.* 53 (2003) 23–69, <https://doi.org/10.1023/A:1025667309714>.
- [5] H. Stoppiglia, G. Dreyfus, R. Dubois, Y. Oussar, Ranking a random feature for variable and feature selection, *J. Mach. Learn. Res.* 3 (2003) 1399–1414, <https://jmlr.org/papers/volume3/stoppiglia03a/stoppiglia03a.pdf>.
- [6] M. Gan, L. Zhang, Iteratively local Fisher score for feature selection, *Appl. Intell.* 51 (2021) 6167–6181, <https://doi.org/10.1007/s10489-020-02141-0>.
- [7] Q. Gu, Z. Li, J. Han, Generalized Fisher score for feature selection, in: *Proceedings of the 27th Conference on Uncertainty in Artificial Intelligence, UAI 2011*, 2011, pp. 266–273, <https://doi.org/10.48550/arXiv.1202.3725>.
- [8] R. Kohavi, G.H. John, Wrappers for feature subset selection, *Artif. Intell.* 97 (1997) 273–324, [https://doi.org/10.1016/S0004-3702\(97\)00043-X](https://doi.org/10.1016/S0004-3702(97)00043-X).
- [9] I.M. Guyon, A. Elisseeff, An introduction to variable and feature selection, *J. Mach. Learn. Res.* 3 (2003) 1157–1182, <https://dl.acm.org/doi/10.5555/944919.944968>.
- [10] G. Dulac-Arnold, L. Denoyer, P. Preux, P. Gallinari, Sequential approaches for learning datum-wise sparse representations, *Mach. Learn.* 89 (2012) 87–122, <https://doi.org/10.1007/s10994-012-5306-7>.
- [11] R. Gaudel, M. Sebag, Feature selection as a one-player game, in: *International Conference on Machine Learning*, 2010, pp. 359–366, <https://icml.cc/Conferences/2010/papers/247.pdf>.
- [12] R. Xu, M. Li, Z. Yang, L. Yang, K. Qiao, Z. Shang, Dynamic feature selection algorithm based on q-learning mechanism, *Appl. Intell.* 51 (2021) 7233–7244, <https://doi.org/10.1007/s10489-021-02257-x>.
- [13] M. Gan, L. Zhang, Q-learning with Fisher score for feature selection of large-scale data sets, in: *Knowledge Science, Engineering and Management*, 2021, pp. 306–318.
- [14] K. Malialis, J. Wang, G. Brooks, G. Frangou, Feature selection as a multiagent coordination problem, *arXiv:1603.05152*, 2016.
- [15] X. Zhao, K. Liu, W. Fan, L. Jiang, X. Zhao, M. Yin, Y. Fu, Simplifying reinforced feature selection via restructured choice strategy of single agent, in: *2020 IEEE International Conference on Data Mining (ICDM)*, 2020, pp. 871–880.
- [16] W. Fan, K. Liu, H. Liu, A. Hariri, D. Dou, Y. Fu, Autogfs: automated group-based feature selection via interactive reinforcement learning, in: *Proceedings of the 2021 SIAM International Conference on Data Mining (SDM)*, 2021, pp. 342–350, <https://epubs.siam.org/doi/pdf/10.1137/1.9781611976700.39>.
- [17] L. Cervante, B. Xue, M. Zhang, L. Shang, Binary particle swarm optimisation for feature selection: a filter based approach, in: *2012 IEEE Congress on Evolutionary Computation*, 2012, pp. 1–8.
- [18] M. Schiezzaro, H. Pedrini, Data feature selection based on artificial bee colony algorithm, *EURASIP J. Image Video Process.* 2013 (2013) 1–8, <https://doi.org/10.1186/1687-5281-2013-47>.
- [19] F. Tan, X. Fu, Y. Zhang, A.G. Bourgeois, A genetic algorithm-based method for feature subset selection, *Soft Comput.* 12 (2008) 111–120, <https://doi.org/10.1007/s00500-007-0193-8>.

- [20] S. Oreski, G. Oreski, Genetic algorithm-based heuristic for feature selection in credit risk assessment, *Expert Syst. Appl.* 41 (2014) 2052–2064, <https://doi.org/10.1016/j.eswa.2013.09.004>.
- [21] M. Deriche, Feature selection using ant colony optimization, in: 2009 6th International Multi-Conference on Systems, Signals and Devices, 2009, pp. 1–4.
- [22] D. Wu, G.G. Wang, Employing reinforcement learning to enhance particle swarm optimization methods, *Eng. Optim.* 54 (2022) 329–348, <https://doi.org/10.1080/0305215X.2020.1867120>.
- [23] S. Sadeg, L. Hamdad, A.R. Remache, M.N. Karech, K. Benatchba, Z. Habbas, Qbso-fs: a reinforcement learning based bee swarm optimization metaheuristic for feature selection, in: *Advances in Computational Intelligence: 15th International Work-Conference on Artificial Neural Networks, IWANN 2019, Gran Canaria, Spain, June 12–14, 2019, Proceedings, Part II* 15, 2019, pp. 785–796.
- [24] Y. Fajri, Wiharto, E. Suryani, Hybrid model feature selection with the bee swarm optimization method and q-learning on the diagnosis of coronary heart disease, *Information* 14 (2022) 15, <https://doi.org/10.3390/info14010015>.
- [25] A.Y. Ng, On feature selection: learning with exponentially many irrelevant features as training examples, in: *Proceedings of the Fifteenth International Conference on Machine Learning*, 1998, pp. 404–412, <http://ai.stanford.edu/~ang/papers/icml98-fs.pdf>.
- [26] C. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292, <https://doi.org/10.1007/BF00992698>.
- [27] R.S. Sutton, A.G. Barto, et al., *Reinforcement Learning: An Introduction*, 1, MIT Press, Cambridge, 1998, <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>.
- [28] H. Samma, C.P. Lim, J.M. Saleh, A new reinforcement learning-based memetic particle swarm optimizer, *Appl. Soft Comput.* 43 (2016) 276–297, <https://doi.org/10.1016/j.asoc.2016.01.006>.
- [29] F. Wang, X. Wang, S. Sun, A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization, *Inf. Sci.* 602 (2022) 298–312, <https://doi.org/10.1016/j.ins.2022.04.053>.
- [30] J. Kennedy, R.C. Eberhart, A discrete binary version of the particle swarm algorithm, in: 1997 IEEE International Conference on Systems, Man, and Cybernetics, in: *Computational Cybernetics and Simulation*, vol. 5, 1997, pp. 4104–4108.
- [31] Q. Gu, Y. Ding, Binary particle swarm optimization based prediction of g-protein-coupled receptor families with feature selection, in: *Proceedings of the First ACM/SIGEVO Summit on Genetic and Evolutionary Computation*, 2009, pp. 171–176.
- [32] D. Moldovan, I. Anghel, T. Cioara, I. Salomie, Adapted binary particle swarm optimization for efficient features selection in the case of imbalanced sensor data, *Appl. Sci.* 10 (2020) 1496, <https://doi.org/10.3390/app10041496>.
- [33] A.-D. Li, B. Xue, M. Zhang, Improved binary particle swarm optimization for feature selection with new initialization and search space reduction strategies, *Appl. Soft Comput.* 106 (2021) 107302, <https://doi.org/10.1016/j.asoc.2021.107302>.
- [34] E. Emary, H.M. Zawbaa, C. Grosan, A.E. Hassenian, Feature subset selection approach by gray-wolf optimization, in: *Afro-European Conference for Industrial Advancement: Proceedings of the First International Afro-European Conference for Industrial Advancement AECIA 2014*, 2015, pp. 1–13.
- [35] M. Mafarja, S. Mirjalili, Whale optimization approaches for wrapper feature selection, *Appl. Soft Comput.* 62 (2018), <https://doi.org/10.1016/j.asoc.2017.11.006>.
- [36] R.Y. Nakamura, L.A. Pereira, K.A. Costa, D. Rodrigues, J.P. Papa, X.-S. Yang, Bba: a binary bat algorithm for feature selection, in: 2012 25th SIBGRAPI Conference on Graphics, Patterns and Images, 2012, pp. 291–297.
- [37] R. Guha, B. Chatterjee, S. Khalid Hassan, S. Ahmed, T. Bhattacharyya, R. Sarkar, Py_fs: a python package for feature selection using meta-heuristic optimization algorithms, in: *Computational Intelligence in Pattern Recognition: Proceedings of CIPR 2021*, 2022, pp. 495–504.
- [38] H. Bai, R. Cheng, Y. Jin, Evolutionary reinforcement learning: a survey, *Intell. Comput.* 2 (2023) 0025, <https://spj.science.org/doi/pdf/10.34133/icomputing.0025>.
- [39] H. Iima, Y. Kuroe, Reinforcement learning through interaction among multiple agents, in: 2006 SICE-ICASE International Joint Conference, 2006, pp. 2457–2462.
- [40] S. Khadka, K. Tumer, Evolution-guided policy gradient in reinforcement learning, *Adv. Neural Inf. Process. Syst.* 31 (2018), https://proceedings.neurips.cc/paper_files/paper/2018/file/85fc37b18c57097425b52fc7afbb6969-Paper.pdf.
- [41] Y. Ma, T.T. Liu, B. Wei, Y. Liu, K. Xu, W. Li, Evolutionary action selection for gradient-based policy learning, in: *International Conference on Neural Information Processing*, 2022, pp. 579–590.